

CODED - Cloud-Optimized Distributed Earth Data

IPFS (InterPlanetary File System) is a viable way to preserve scientific data. A **Zarr** or **Icechunk** dataset converts to content-addressed **IPFS** in one command, pins on a service like Filebase, and re-opens straight into **xarray** - takedown-resistant, verifiable, and performant enough for real work.

Cory Levinson • Rich Signell • Brian Terry

★ An ESIP Funding Friday Project

How we built this →

We chatted with an OpenClaw AI agent over Telegram - it ran the benchmarks, spun nodes up and down, and drafted these figures.

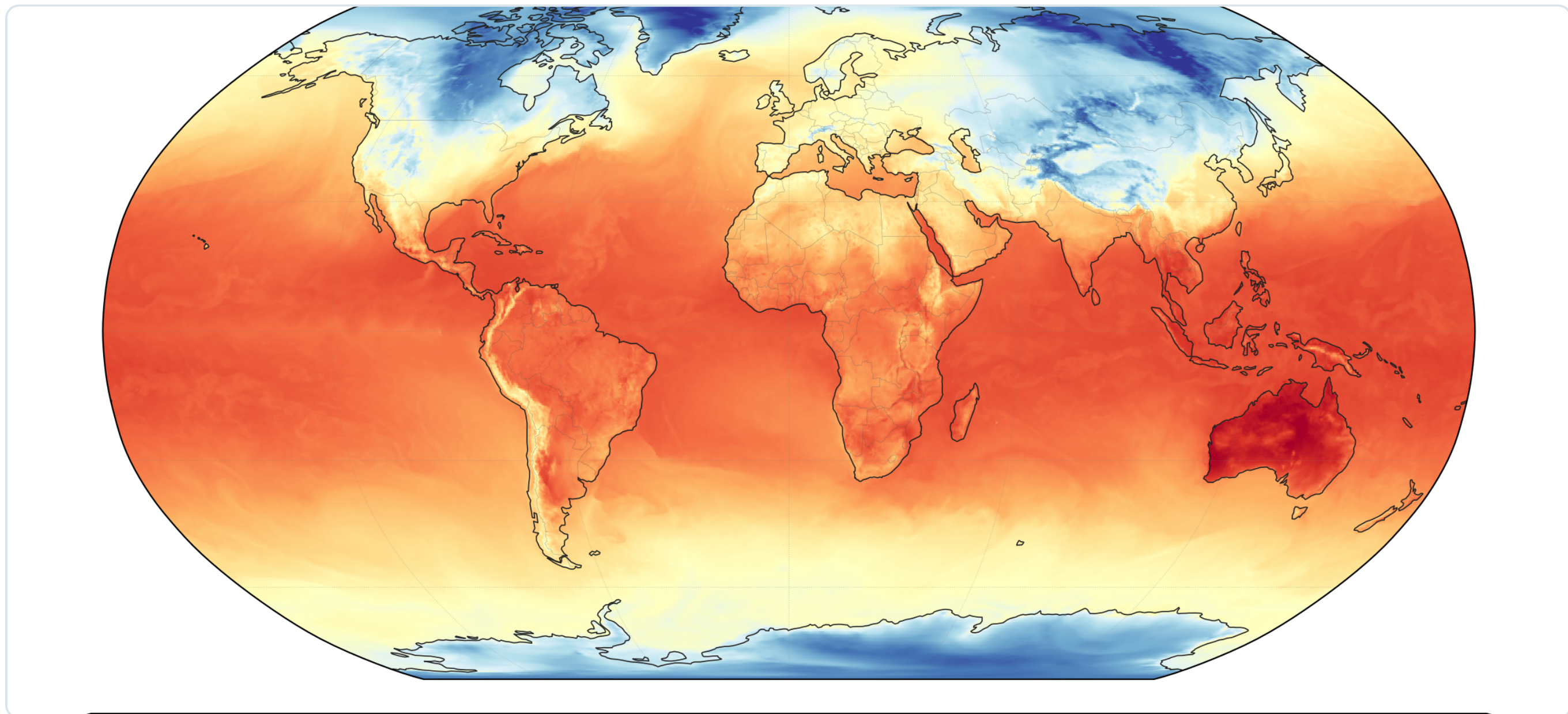


Why preserve on IPFS?

Important environmental datasets can vanish when an institution reorganizes, loses funding, or is told to take them down. **IPFS** stores data by **content hash**, not by owner or location - so a dataset stays reachable and verifiable no matter who hosts it.

The good news: the cloud-optimized formats the community already uses - Zarr and Icechunk - drop onto IPFS as-is, and open right back up in xarray.

Live proof • ERA5 straight from IPFS



Live proof: this global ERA5 2 m-temperature field was opened **straight from IPFS** - `icechunk.http_storage("ipfs.io/ipfs/<CID>")` → `xr.open_zarr` - in **<1 s**. Content-addressed data, real analysis, no download step.

Step 1 • Preserve your dataset - CID on Filebase

Have an existing **Zarr** or **Icechunk** store you can't afford to lose? Add it to IPFS for a content identifier (CID), then pin that CID on **Filebase** (an IPFS + Filecoin pinning service, free to 5 GB / 500 pins) - or **any public IPFS node** - so it stays available, verifiable, and independent of any single institution.

```
# add your existing store to IPFS → one root CID
cid=$(ipfs add -rQ --cid-version=1 my.icechunk/)
# pin that CID on Filebase (IPFS + Filecoin) - stays alive
# independent of your own node. Same CID, now persistent.
```

The **CID is a hash of the data** — tamper-evident and location-independent. Our live ERA5 store:

`bafybeiecltl3mtags2i3jeumxe5c7zuvj2r76zxwxg6tfljiouvvywqzbaq`

One CID = the whole queryable store. Re-chunked to 100 blocks (5 timesteps each) to fit the free-tier pin budget - **data byte-identical** (maxdiff 0.0).

Step 2 • Icechunk reads IPFS - no adapter

`Icechunk 2.0.5` ships `http_storage(base_url=...)`, a read-only store over HTTP. Point it at an IPFS gateway path and the whole repo opens in `xarray` - the on-disk layout survives `ipfs add -r` untouched. **Zero adapter code.**

```
import icechunk, xarray as xr

GATEWAY = "https://ipfs.io"
CID     = "bafybeiecltl3mtags2i3jeumxe5c7zuvj2r76zxwxg6tfljiouvvywqzbaq"

# read-only Icechunk repo, straight from a CID over IPFS
storage = icechunk.http_storage(f"{GATEWAY}/ipfs/{CID}")
repo    = icechunk.Repository.open(storage)
ds      = xr.open_zarr(repo.readonly_session("main").store,
                      consolidated=False, chunks={})
```

Old CID → old snapshot (reproducibility). Dedup across versions comes free - edits reuse unchanged blocks.

... and it's performant enough

We read the **same 2.08 GB ERA5** temperature slice (100 Icechunk blocks) over IPFS via `http_storage`, against a **native Icechunk-on-S3** baseline. Every path returns a bit-identical mean = **286.5062 K**.

	~8 s	~260
	warm / local read (beats S3-Icechunk)	MB/s once cached locally
Read path	Same-region	Cross-region
IPFS local pin / warm	~8 s	~8 s
Icechunk-on-S3 (baseline)	~14 s	~14 s
IPFS cold, first pull	~75 s	~101 s

Only the first cross-network pull of the full 2 GB is slow; it caches away and repeat reads beat S3. Pin near your readers (or co-locate a gateway) and IPFS delivers S3-class throughput.

What we verified

- **It survives your node.** With our EC2 node off, the CID still resolved & opened via public gateways (Filebase, ipfs.io, dweb.link) - Filebase kept serving it.
- **The data is exactly the data.** The CID is a hash of the bytes - tamper-evident, byte-identical across every read (maxdiff 0.0).
- **No new tooling.** `ipfs add` to publish, `icechunk.http_storage` to read - the formats you already use, unchanged.

Bottom Line

IPFS is a practical preservation layer for cloud-native Earth data - today. Convert in one command, pin it on a cooperating organization's IPFS node or a commercial pinning service, re-open in `xarray`. Your data becomes content-addressed, takedown-resistant, and reproducible - and it reads back fast enough for real analysis.

Live demo & notebooks: github.com/ESIPFed/coded-blog (see **ipfs-agent/**)

Scan the codes below to run it yourself →

Acknowledgements: This work was carried out by **chatting with an OpenClaw AI agent over Telegram** (AWS VM, Claude via Amazon Bedrock) - the team steered; the agent ran the benchmarks, provisioned & tore down nodes, and drafted the figures. Thanks to AWS for the ESIP Credits!



Demo notebook
read ERA5 from IPFS



Final report
full write-up